coverage.py vs. pytest-cov: How They Relate and When to Use Which

SE 413/513 Software Testing · Demo: demos/coverage_vs_pytest_cov

Dr. Kamrul Hasan, Grand Valley State University

Learning objectives

1

Relate

Explain how coverage.py and pytest-cov relate, and why they report the same numbers.

2

Translate

Turn a pytest-cov command into the equivalent coverage.py commands, and back.

3

Choose

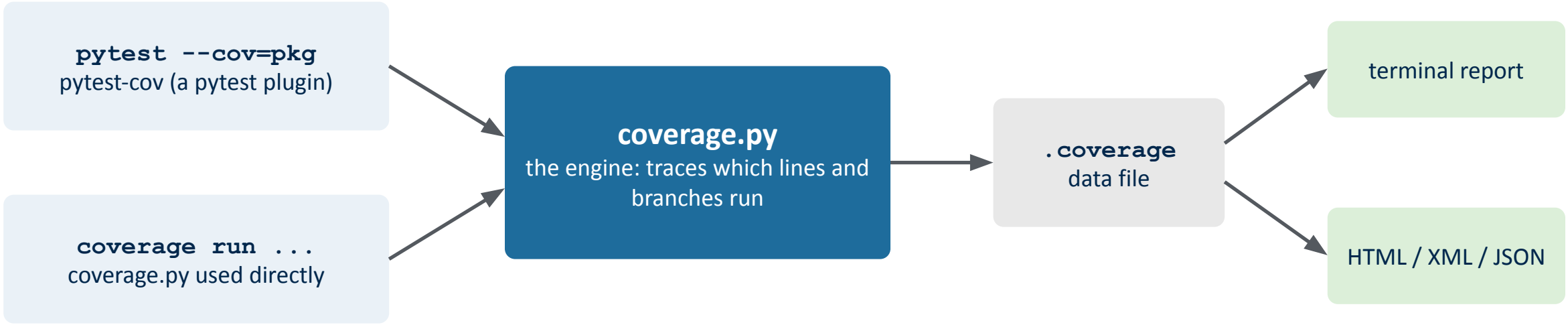
Pick the right tool: everyday tests, parallel runs, CI, scripts, combining runs.

4

Avoid

Spot three pitfalls: test files in the total, a wrong --cov target, stale data.

One engine, one plugin



coverage.py measures any Python program: a test run, a script, a command-line tool.

pytest-cov switches coverage.py on and off inside pytest and prints the report at the end.

Key idea

Not competitors: pytest-cov depends on coverage.py. Same engine, so the same numbers.

What is in the demo

LIVE DEMO 1
setup + look around

File	What it is
`tempconv/temps.py`	The code under test (don't reformat: report line numbers refer to it)
`tests/test_temps.py`	A deliberately thin suite with two tests
`report.py`	A small command-line tool, not a test; pytest never runs it
`coverage_settings.toml`	Saved coverage settings (used on the shared-config slide)

Setup: run everything from this folder

```
cd demos/coverage_vs_pytest_cov

python -m pip install \
  pytest pytest-cov \
  pytest-xdist
```

Start each experiment clean

`coverage erase` deletes the previous .coverage data.

The code under test

tempconv/temps.py

```
 1  def to_fahrenheit(c):
 2      return c * 9 / 5 + 32
 3
 4
 5  def describe(c):
 6      if c < 0:
 7          return "freezing"
 8      elif c < 25:
 9          return "mild"
10      return "hot"
```

Red: never run by the tests (lines 7, 10). **Green**: the only describe() path tested.

tests/test_temps.py

```
def test_boiling():
    assert to_fahrenheit(100) == 212

def test_mild():
    assert describe(20) == "mild"
```

report.py (a tool, not a test)

```
for arg in sys.argv[1:]:
    c = float(arg)
    f = to_fahrenheit(c)
    print(f"{c} C = {f} F ({describe(c)})")
```

Run by hand: python report.py -5 30

Same measurement, two ways

LIVE DEMO 2
run both commands

pytest-cov: one command

```
pytest --cov=tempconv --cov-branch \
       --cov-report=term-missing
```

coverage.py: measure, then report

```
coverage run --branch --source=tempconv -m pytest
coverage report -m
```

Both print:

```
Name                 Stmts   Miss  Branch  BrPart  Cover   Missing
tempconv/temps.py        8      2       4       2    67%   7, 10
```

pytest-cov option	coverage.py equivalent	pytest-cov option	coverage.py equivalent
`--cov=PKG`	`run --source=PKG`	`--cov-report=html`	`coverage html`
`--cov-branch`	`run --branch`	`--cov-report=xml`	`coverage xml`
`--cov-report=term-missing`	`report -m`	`--cov-fail-under=N`	`report --fail-under=N`

Try it: the same number, two ways

TRY IT 1
students · 6 min

1. Measure the suite with pytest-cov. Note Cover and Missing.

```
coverage erase
pytest --cov=tempconv --cov-branch \
  --cov-report=term-missing
```

2. Measure it again with coverage.py directly. Do the numbers match?

```
coverage erase
coverage run --branch --source=tempconv -m pytest
coverage report -m
```

3. Create tests/test_more.py with tests for a freezing and a hot temperature. Re-run step 1. Then delete the file.

```
def test_freezing():
    assert describe(-5) == "freezing"
# ...and test_hot()
rm tests/test_more.py   # afterwards
```

Done early? Which line of temps.py does each new test cover? Check with coverage html.

They share a data file and a config

LIVE DEMO 3
reuse data + config

1 One data file

```
pytest --cov=tempconv --cov-branch
ls -a                # .coverage is here
coverage report -m   # same 67%, no re-run
coverage html        # open htmlcov/index.html
```

Measure once, then report as many ways as you like.

2 One config (pyproject.toml)

```
[tool.coverage.run]
branch = true
source = ["tempconv"]

[tool.coverage.report]
show_missing = true
fail_under = 80
```

The demo stores this in coverage_settings.toml so it applies only when passed explicitly.

```
pytest --cov --cov-config=coverage_settings.toml
  FAIL Required test coverage of 80.0% not reached. Total coverage: 66.67%      exit 1
coverage report --rcfile=coverage_settings.toml
  Coverage failure: total of 67 is less than fail-under=80                      exit 2
```

When to use coverage.py directly

LIVE DEMO 4
script, then --append

A. Code pytest does not run

```
coverage erase
coverage run --branch --source=tempconv \
             report.py -5 30
coverage report -m
```

```
-5.0 C = 23.0 F (freezing)
30.0 C = 86.0 F (hot)
tempconv/temps.py    8    1    4    1    83%    9
```

B. Adding runs together

```
coverage run --append --branch \
    --source=tempconv -m pytest
coverage report -m
```

```
tempconv/temps.py    8    0    4    0  100%
```

--append adds to the existing data instead of replacing it.

67%

tests only (missing 7, 10)

83%

script only (missing 9)

100%

script + tests (--append)

Try it: a script, measured

TRY IT 2
students · 5 min

1. Measure report.py alone with one value. Which lines are missing?

```
coverage erase
coverage run --branch --source=tempconv \
  report.py 20
coverage report -m
```

2. Find the fewest arguments to report.py that reach 100% with no tests at all.

```
coverage erase
coverage run --branch --source=tempconv \
  report.py <your values>
coverage report -m
```

3. Predict, then check: what does report.py 0 25 report? Why?

```
coverage erase
coverage run --branch --source=tempconv \
  report.py 0 25
coverage report -m
```

When to use pytest-cov: parallel runs and CI

LIVE DEMO 5
-n 2, then CI gate

Parallel tests with pytest-xdist (-n 2)

```
pytest -n 2 --cov=tempconv --cov-branch
```

```
coverage run --branch --source=tempconv -m pytest -n 2
coverage report
```

67%

pytest -n 2 --cov: each worker measured

0%

coverage run: only the main process seen

```
CoverageWarning: No data was collected.
tempconv/temps.py    8    8    4    0    0%
```

The tests passed, and coverage report still exits 0, so a careless CI script would not notice.

A coverage gate in CI

```
pytest --cov=tempconv \
  --cov-branch \
  --cov-fail-under=90
echo $?
```

```
FAIL Required test
  coverage of 90% not
  reached. Total
  coverage: 66.67%
2 passed
1
```

Tests pass, the command fails (exit 1), so the CI build fails.

Try it: parallel runs and a gate

TRY IT 3
students · 4 min

1 Run the tests in parallel with pytest-cov. Record Cover and the exit code.

```
coverage erase
pytest -n 2 --cov=tempconv --cov-branch \
  --cov-fail-under=60; echo $?
```

2 Do the same with coverage.py directly, with the same gate.

```
coverage erase
coverage run --branch --source=tempconv -m pytest -n 2
coverage report --fail-under=60; echo $?
```

Done early? Without --fail-under, what exit code does step 2's coverage report give? What does that mean for a CI script?

Three pitfalls

LIVE DEMO 6
reproduce the pitfalls

! Forgetting --source

Test files almost always run completely, so they inflate the total.

```
$ coverage run -m pytest
$ coverage report
temps.py         8   2   75%
test_temps.py    5   0  100%
TOTAL           13   2   85%
```

! A wrong --cov target

--cov= names the code measured, not the tests. A typo gives only warnings, green tests, and exit 0.

```
$ pytest --cov=tempconv.nothere
Module tempconv.nothere
  was never imported.
No data was collected.
2 passed
```

! Stale data

--append keeps old results. Run coverage erase before a fresh measurement, or earlier experiments leak in.

```
$ coverage erase
$ coverage run ...
$ coverage report
```

Try it: make the pitfalls happen

TRY IT 4
students · 4 min

1. Leave out --source. What is TOTAL, and why is it wrong?

```
coverage erase
coverage run -m pytest
coverage report
```

2. Misspell the package. Do the tests pass? What is the exit code?

```
coverage erase
pytest --cov=tempcon; echo $?
```

3. Append a second script run. What does the report say, and is that what you meant to measure?

```
coverage erase
coverage run --branch --source=tempconv \
  report.py -5 30
coverage run --append --branch \
  --source=tempconv report.py 20
coverage report
```

Quick comparison

Capability	pytest-cov	coverage.py directly
What it can measure	A pytest run only	Any Python process
Commands	One (pytest --cov)	Two or more (run, then report/html)
Parallel tests (pytest -n)	Handled automatically	Needs extra setup (else 0%)
Combining separate runs	--cov-append	--append, coverage combine
Re-report without re-running	No	Yes (coverage report / html)
Scripts, unittest, manual sessions	No	Yes
Typical use	Everyday testing and CI	Investigation and special cases

Practice question

TRY IT
think, then vote · 2 min

You just ran `coverage erase` and `coverage run --branch --source=tempconv report.py -5 30`, which reports 83% with line 9 missing.

Which single next command, followed by coverage report, shows 100%?

A `coverage run --branch --source=tempconv -m pytest`

B `pytest --cov=tempconv --cov-branch`

C `coverage run --append --branch --source=tempconv -m pytest`

D `coverage report -m`

Answer: C

LIVE DEMO 7
verify all 4 options

C `coverage run --append --branch --source=tempconv -m pytest` → 100%

× **A, B** Each replaces the data file, so the report shows the tests alone: 67%, missing 7 and 10.

× **D** Re-reports the existing data and still shows 83%.

\+ **Also** pytest-cov's own --cov-append option gives 100% here too.

Key takeaways

1. coverage.py is the engine; pytest-cov is a plugin that drives it from pytest.
2. Same engine, same .coverage file, same config: same numbers.
3. Default to pytest --cov, especially for parallel runs and CI.
4. Use coverage.py directly for non-pytest code, combining runs, and re-reporting.
5. Always measure branches, and only your own code.